> **TdeA PUNTO.TDEA**

MANUAL DE DESPLIEGUE Y CONFIGURACIÓN

Control documental

Campo	Valor
Código	PTDA-MD-001
Título	Manual de despliegue y configuración de Punto TdeA
Versión	1.0
Fecha	Noviembre 2026
Estado	Vigente
Sistema	Punto TdeA, atención por WhatsApp
Relacionado	Manual técnico PTDA-MT-001

1. Propósito y alcance

Este manual explica cómo instalar y desplegar Punto TdeA en cualquier servidor: Render, Azure, AWS o un servidor físico/VPS propio. Detalla la configuración de variables, el orden de arranque, la validación y el rollback.

No describe la arquitectura interna ni el funcionamiento de los módulos: eso está en el manual técnico (PTDA-MT-001). Si este manual y el código discrepan, manda el código.

2. Requisitos previos

Los detalles de hardware, software y red están en la sección 3 del manual técnico. En resumen:

Rol	vCPU	RAM	Disco
Aplicación	2	4 GB	30 GB
Datos	2	4 GB	30 GB + 32 GB PostgreSQL
Redis	Comparte la aplicación	1 GB	No guarda operación

- Linux x64 (Ubuntu Server 24.04 LTS u equivalente).
- Docker Engine y Compose v2.
- PostgreSQL 15 y Redis 7 (locales o administrados).
- Dominio público con certificado TLS en la entrada.
- Puertos: 443 público; 5432 (PostgreSQL) y 6379 (Redis) solo en la red privada.
- La aplicación sale hacia `graph.facebook.com:443` (Meta).

3. Configuración

La aplicación no debe arrancar en operación sin estas variables. Lo marcado como secreto no se commitea ni se pega en este manual.

Variable	Secreto	Para qué
DJANGO_DEBUG	No	`false` en operación.
DJANGO_SECRET_KEY	Sí	Obligatoria si DEBUG es `false`.
DJANGO_ALLOWED_HOSTS	No	Dominio público, separado por comas. Render usa `RENDER_EXTERNAL_HOSTNAME` si no se define.
DJANGO_SECURE_SSL_REDIRECT	No	Redirige a HTTPS con DEBUG=`false`; se pone en `false` para probar en local con el test client.
DATABASE_URL o DB_*	Sí la clave	PostgreSQL. Si faltan, cae a SQLite (no es operación).
REDIS_URL	No	Canal de WebSocket. Sin él, capa en memoria: no sirve con más de un proceso.
ENABLE_INACTIVITY_SCHEDULER	No	`true` incrusta el ciclo 5+2 de inactividad del chatbot en el proceso web.
ENABLE_CAMPAIGN_SCHEDULER	No	`true` incrusta el envío de campañas programadas vencidas en el proceso web.
WIDGET_ALLOWED_ORIGINS	No	Orígenes CORS permitidos para el widget web.
META_ACCESS_TOKEN	Sí	Enviar y leer plantillas.
WHATSAPP_PHONE_NUMBER_ID	No	Número de Cloud API.
WHATSAPP_BUSINESS_ACCOUNT_ID	No	Crear plantillas. Sin esto, "Enviar a Meta" no puede.
META_APP_SECRET	Sí	Firma del webhook.
META_VERIFY_TOKEN	Sí	Verificación GET del webhook.
INTERNAL_API_TOKEN	Sí	La UI de campañas lee segmentos con este token.
CASES_WEB_WEBHOOK_TOKEN	Sí	Integración externa de ingesta web de cases (`X-Integration-Token`). No está en `render.yaml`.
META_ADS_ACCESS_TOKEN	Sí	Anuncios en WhatsApp Status (Marketing API). No es el token de Cloud API.
MEDIA_UPLOADS_ENABLED	No	Declarada en settings pero hoy no se consulta: no bloquea subidas por sí sola.

Python no lee un archivo `.env` por su cuenta: lo sustituye el compose o el entorno del host. Después de cambiar una variable hay que **recrear** el proceso; un reinicio que no recarga el entorno deja el valor viejo.

4. Instalación local (desarrollo)

```
docker compose up -d --build
docker compose exec web python manage.py migrate
docker compose exec web python manage.py createsuperuser
```

Sin Docker, con el entorno del repositorio y sin `DB_HOST`, Django usa SQLite. Alcanza para pruebas de pantalla, no representa la operación.

```
python -m venv venv
./venv/bin/pip install -r requirements.txt
./venv/bin/python manage.py migrate
./venv/bin/daphne -b 127.0.0.1 -p 8000 PuntoTdeA.asgi:application
```

5. Despliegue

5.1 Requisitos comunes

- La imagen del release se construye con el `Dockerfile`. El build ya corre `collectstatic`; con `DEBUG=false`, whitenoise sirve los estáticos.
- `entrypoint.sh` aplica migraciones con reintentos, crea el superusuario inicial y levanta Daphne en `$PORT` (8000 por defecto).
- PostgreSQL 15 y Redis 7 en la red privada; solo la entrada web es pública.
- Los schedulers de inactividad y campañas pueden ir **incrustados en el web** (`ENABLE_INACTIVITY_SCHEDULER=true` y `ENABLE_CAMPAIGN_SCHEDULER=true`) o como procesos aparte.

5.2 Orden de arranque

01 **Red**: abrir solo los puertos de la sección 2. El 5432 y el 6379 quedan privados.

02 **Datos**: instalar PostgreSQL 15, montar el disco de datos y crear una base y un usuario de la aplicación. No usar el superusuario `postgres` desde Django.

03 **Redis**: levantarlo en la red de la aplicación.

04 **Secretos**: cargarlos en el entorno del host. Ninguno va en la imagen.

05 **Imagen**: construir la imagen del release.

06 **Migraciones**: el arranque las aplica y aborta si no puede. No se sirve tráfico con un esquema viejo.

07 **Procesos**: Daphne en el puerto interno. Los schedulers van incrustados (flags de arriba) o, en un servidor propio, como servicios de systemd/supervisor o cron:

```
python manage.py close_inactive_chats --interval 60
python manage.py run_scheduled_campaigns --interval 30
```

08 **Meta**: apuntar el webhook a `/api/whatsapp/webhook/` con el token de verificación. Una sola URL.

5.3 Render

`render.yaml` es el Blueprint: web (Docker) + Key Value (Valkey) + Postgres. El deploy arranca solo si CI está en verde (`autoDeployTrigger: checksPass`). Los secretos se cargan en el panel (`sync: false`). El `entrypoint.sh` corre migraciones + superusuario + Daphne; no usar `dockerCommand` (el CMD del Dockerfile ya hace el arranque). En el plan free el servicio duerme a los 15 min, Postgres expira a los 30 días y no hay disco persistente. Para pasar a pago, cambia `plan: starter` (web), `basic-256mb` (Postgres) y `256mb` (Key Value). Ver `docs/despliegue-render.md`.

5.4 Azure

Tres caminos, de menos a más administración:

- **Container Apps**: despliega la imagen del `Dockerfile`, con Azure Database for PostgreSQL Flexible Server y Azure Cache for Redis. Las variables van en la Container App; el health check apunta a `/api/whatsapp/health/`. Escala a 0 para ahorrar (la primera petición tarda).
- **App Service (contenedor Linux)**: similar; expón el puerto 8000 con `WEBSITES_PORT=8000` y usa el endpoint de health.
- **Máquina virtual (Ubuntu)**: Docker Compose + systemd + Nginx/Caddy con TLS, con Postgres y Redis en la misma VM o en servicio administrado.

La media persistente requiere un volumen: en Container Apps/App Service monta Azure Files en `/app/media`; en VM, un disco aparte.

5.5 AWS

- **ECS/Fargate**: tarea con la imagen, RDS PostgreSQL, ElastiCache Redis y un ALB que termine HTTPS y apunte el health a `/api/whatsapp/health/`. Los secretos van en Secrets Manager o en las variables de la tarea. La media persistente se monta desde EFS en `/app/media`.
- **EC2**: Docker Compose + systemd + Nginx/Caddy, con RDS y ElastiCache o con Postgres/Redis locales.

5.6 Servidor físico o VPS

Es el camino del compose de desarrollo, con reverse proxy y TLS:

```
docker compose up -d --build
docker compose exec web python manage.py migrate
docker compose exec web python manage.py createsuperuser
```

Nginx o Caddy delante del 8000, con certificado. Postgres y Redis en la misma máquina o en otra interna. Monta `media/` y el disco de Postgres como volúmenes. Los schedulers pueden ir incrustados (los flags de arriba) o como systemd timers.

6. Validación

Prueba	Resultado esperado
Login de un asesor	Entra al dashboard y el menú muestra solo sus módulos.
GET /api/whatsapp/health/	Responde sano.
Bandeja	Carga y el WebSocket no se cae al minuto.
Puerto 5432 desde internet	Sin respuesta.
Campaña programada vencida	Se envía sin levantar un proceso aparte (scheduler incrustado).
Chat inactivo	Avisa a los 5 min y cierra a los 7 si no hay respuesta.
Cierre de un caso de prueba	El caso queda cerrado aunque Meta rechace el aviso.

7. Rollback

- Antes de migrar, `pg_dump` de la base. Sin ese archivo no hay vuelta atrás segura.
- Conservar la imagen anterior. Si el health falla, volver a esa imagen.
- Volver el código sin restaurar la base solo es válido si las migraciones nuevas no rompen el código viejo. Si lo rompen, se restaura el dump.
- El rollback depende del proveedor: en Render se redeploya el commit anterior; en Azure o AWS se vuelve a la versión anterior del contenedor; en servidor propio se reintroduce la imagen anterior.
- Los adjuntos están en disco, en `media/`. Un contenedor nuevo sin ese volumen los pierde aunque la base vuelva.

8. Operación

- **Migraciones**: el arranque las aplica con reintentos; a la décima falla, el contenedor no queda sirviendo.
- **Schedulers**: si van incrustados, corren mientras el proceso web está despierto. En el plan free de Render el contenedor duerme a los 15 min, así que los ciclos pueden retrasarse.
- **Health**: `/api/whatsapp/health/` exige sesión staff y responde 302 al login; los proveedores aceptan 2xx/3xx como sanos.
- **Backups**: `pg_dump` diario, guardado fuera del disco de datos. Redis no se respalda. `media/` se copia junto con el dump si hay adjuntos que no se pueden perder.
- **Runbooks**:
  - La bandeja no recibe WhatsApp: URL única del webhook, `META_VERIFY_TOKEN` y `WHATSAPP_PHONE_NUMBER_ID`.
  - El aviso amarillo de plantillas: faltan `META_ACCESS_TOKEN` o `WHATSAPP_BUSINESS_ACCOUNT_ID`.
  - Una campaña programada no sale: revisar `ENABLE_CAMPAIGN_SCHEDULER` o el proceso `run_scheduled_campaigns`.
  - 403 en un módulo: falta el permiso `access_` del módulo. Cerrar sesión y volver a entrar.
  - Adjuntos desaparecen al redesplegar: `media/` no está en un volumen.

9. Referencias

- Manual técnico: PTDA-MT-001 (arquitectura, requisitos, datos, operación).
- Despliegue en Render: `docs/despliegue-render.md`.
- Integración Meta: `docs/whatsapp-meta.md`, `docs/guia-uso-para-otros-modulos.md`.
- Pruebas de la integración Meta: `docs/pruebas-integracion-meta.md`.